The P Versus NP Problem

Kabir Kanha Arora

December 11, 2018

Abstract

Today, I want to talk about one of the deepest unanswered questions in Computer Science and perhaps, in *all* of mathematics. (Fortnow, 2009) A problem called **P versus NP.** And I want to talk about how ideas from computing show up in the everyday world around us, and how problems as seemingly unconnected as *protein folding* and making up *crossword puzzles* share a common core difficulty that turns out to be a lot like *Sudoku*. We also move on the discuss what P and NP stand for and what the relationship between them is. We also discuss the consequences of establishing any sort of relationship between them, and end the paper with an examination of classes other than P and NP.

1 Introduction

In 2000, the Clay Institute offered *$1,000,000* each for seven key problems in Math - **the Millenial Prize Problems:**

1. Birch and Swinnerton-Dyer Conjecture
2. Hodge Conjecture
3. Navier-Stokes existence and smoothness
4. P versus NP problem
5. Poincaré Conjecture *(Solved)*
6. Riemann hypothesis

7. Yang-Mills existence and mass gap

These are profound and difficult problems and for most of them, it takes a lot of specialised knowledge to even understand the question. Of the seven problems, P versus NP was both the most recently conceived (in 1971) and by far the easiest one to understand and explain.

2 So what is the P vs NP question?

Well, back in the 1970s computer scientists were busily figuring out how to program their retro-fabulous, cabinet-sized computers to solve all the world's problems. Sometimes the first program anyone could think of for a particular problem would be unworkably slow but then, over time people would come up with clever ways to make it faster, or at least, that happened for some problems. For others, nobody was coming up with faster programs. To get a grip on the situation they started sorting the problems into classes based on how fast the program can solve them. For problems like multiplication they had really fast programs, and for others, like playing absolutely perfect chess they figured out that there just was no fast program. But for a bunch in-between they weren't sure whether there was a fast way to do it. So, they kept trying. This is where P and NP come in. Skipping a ton of details for a second, **P** is a class that basically includes *all the problems that can be solved by a reasonably fast program*, like multiplication or alphabetizing a list of names. And then, around and including P we sort of discovered a class called **NP** That's all the problems where, *if you're given a correct solution you can at least check it in a reasonable amount of time.* NP was totally maddening, because it contained lots of important problems like vehicle routing and scheduling and circuit design and databases. And often, we get lucky and find that an NP problem was actually a part of P and we'd have our fast program. But, for a lot of them that didn't seem to be happening. So people started to wonder whether everything in NP would turn out to be in P or if there were some NP problems that were truly harder than the ones in P. **That's the P vs. NP question.**

3 Explanation

*The **P versus NP problem** is a major unsolved problem in computer science. It asks whether every problem whose solution can be quickly verified (technically, verified in polynomial time) can also be solved quickly (again, in polynomial time)* (Ladner, 1975).
The informal term quickly, used above, means the existence of an algorithm solving the task that runs in polynomial time, such that the time to complete the task varies as a polynomial function on the size of the input to the algorithm (as opposed to, say, exponential time).
In general, the class of questions for which some or the other algorithm can provide an answer within polynomial time is called 'class P' or simply, just **'P'**. However, there are some questions, for which there is no known way of finding an answer quickly, but if we are given the answer, it can be verified in polynomial time. Such a class of questions is called 'Class NP', or simply **'NP'**, which is short for *"Non-deterministic Polynomial Time"*.

4 A few real-life examples

I like to think of the problems in NP as being basically like "puzzles" because I think what makes a puzzle a puzzle is that it's a problem where you can give away the answer, and that's what NP means. Like with Sudoku. Sudoku puzzles can take a long time to solve but if I give you a solved Sudoku grid checking it for mistakes is pretty quick.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 1: A sample Sudoku grid

Outside of NP there are problems where it's hard to even check an answer. Like, *what's the best move to make in this chess game?* I could tell you the answer but how would you know whether I'm right? Well, you wouldn't, because finding out requires a calculation so enormous that there's a pretty good argument we'll never be able to build a computer that could do it. To me, that's not a very good puzzle. It's practically impossible to know whether you've solved it.

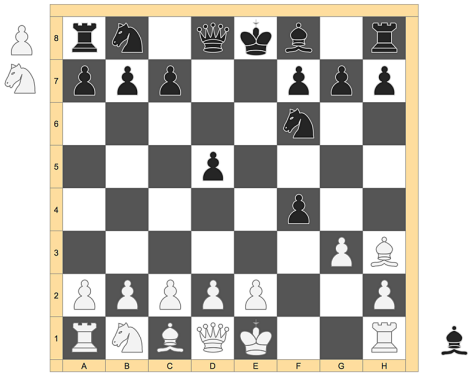

Figure 2: An ongoing Chess game

On the other side, are all the reasonable, solvable puzzles in P. These are clearly also in NP because one way to check an answer is to go through the process of finding it yourself. Like, if I were to tell you that the answer to **51 x 3** is **153**, how would you check whether I'm right? You'd probably just multiply 51 by 3 yourself because it's fast to do it, but Sudoku is different, or at least, we think it is. It seems like solving a Sudoku grid is a lot harder than checking a solution, but in fact nobody's been able to prove it yet. As far as we know, there could be a clever way of playing Sudoku, much much faster. So that's the question: Does being able to quickly recognize correct answers mean there's also a quick way to find them? Nobody knows for sure, but either way, figuring out exactly how this works would teach us something important about the nature of computation.

5 Some important points

5.1 So what's the big deal

You might be thinking: hey, Sudoku is tough and all but it's not that hard. What's the big deal? Well, we're really talking about how the difficulty scales up as you make the problem bigger and bigger. Like, how much harder is a 100x100 Sudoku grid than a standard 9x9 grid? We've been making computers exponentially faster as time goes on, so, for problems that don't get exponentially harder as they get bigger all we have to do is wait for computers to get more powerful and then, even huge versions of those problems will be easy to solve by a computer. Like, multiplication problems are pretty easy for computers, even with enormous numbers. As the numbers get bigger, multiplying them just doesn't get harder very fast. These days, the phone is what would have been referred to in the 1970s as a "supercomputer". And you'd have to make up truly huge multiplication problems to stand up to all the computational power we've got now. Lots of familiar puzzles like mazes and Rubik's cubes are in the same camp. Hard for people, but easy work for computers. And then there's Sudoku. **Computers can usually solve a 9x9 grid in a few milliseconds even though humans find them challenging. But as you make the grid bigger the problem just gets really hard, rapidly getting out of reach for even the most powerful computers.**

5.2 P stands for "Polynomial time".

In P the number of steps you have to do to solve a problem, and therefore the amount of time that it takes, is some polynomial function of its size. "*Polynomial*" is a mishmash of Greek and Latin meaning "*many names*", which is, regrettably a pretty typical example of Math's flair for unhelpful terminology. Anyway, polynomials are functions involving n or n^2 or n to other powers like these, but importantly they're not exponential functions like 2^n, which gets to be a ton of steps really fast as n goes up, a lot quicker than n^2. So that's P, it's problems like mazes and multiplication where the number of steps required isn't that bad compared to the size of the problem. NP is all about polynomial time checking. NP stands for "Non deterministic polynomial time", which, being math terminology is almost a mean spirited way of saying that

if you had a zillion computers then you could check all possible answers at the same time. You could find a correct solution in polynomial time. We're actually talking about the number of steps required to solve a problem in the worst case scenario. Like, how many steps does it take to unscramble a Rubik's cube? Well, in the best case, it takes one step (or even none), but it can get a whole lot worse. Similarly, some 9x9 Sudokus are harder than others. People also look at things like the best case and the average case, but worst case analysis is what we know the most about.

5.3 Everybody thinks NP contains more problems than P

It's just that we haven't been able to prove it. The bad news for fast solutions came in the early 1970s where complexity researches realized that dozens of those NP problems they were struggling with were essentially all the same problem! (with some easy polynomial time complications thrown in here and there) These are call "NP-complete" problems, and since that first batch in the 70s we've added Sudoku and protein folding, and problems underlying puzzles and games like Battleship, *FreeCell*, *Master Mind*, *Tetris*, *Minesweeper*, and making up *crossword* puzzles. Even classic video games like *Super Mario Bros.* and *Metroid* turn out to be connected to NP complete-level traversal problems. NP-complete is yet another Math phrase meaning that these problems include all the really hard parts of every NP problem. A fast program for solving any NP-complete problem could be used to solve every problem in NP. The whole class would instantly collapse. So yeah, amazingly, Sudoku is hard because it involves, literally, the same NP-complete task that makes protein folding hard. If you come up with a profoundly faster way to play Sudoku let somebody know, okay? because fast protein folding would **help us cure cancer**. But the fact that a bunch of smart people have all been unsuccessful coming up with fast programs to solve what turned out to be, essentially, the same problem, looks like pretty good clue that the fast programs just aren't out there.

6 Does P mean "easy"?

So far, we have continued the discussion with the underlying presupposition that P means "**easy**" and NP means "**hard**", and this is known as *Cobham's*

thesis. It seems as though this is a reasonably accurate and commonplace assumption. However, there are some things that need a closer inspection. For one, it is not always true in practice. In theory, a P algorithm may have extremely large constant factors or exponents, making it impractical. Or, even when a problem is NP-complete, and even if P $\neq$ Q, there may still exist some effective approaches to tackling the problem in practice. We know that there are algorithms for many NP-complete problems, such as the *knapsack problem*, the *travelling salesman problem* and the *Boolean satisfiability problem*, that we can solve to optimality in many real world cases and in not too much time. The **average time taken vs. problem size** for such algorithms is much lower than what we would have expected. An example is linear programming, which works surprisingly well in practice, despite having exponential worst-case time complexity it runs on par with the best known polynomial-time algorithms.

7 So is P = NP or P $\neq$ NP?

7.1 Reasons to believe P $\neq$ NP

According to polls, most computer scientists believe that P $\neq$ NP. A key reason for this belief is that after decades of studying these problems no one has been able to find a polynomial-time algorithm for any of more than 3000 important known NP-complete problems. (Gasarch, 2002) (Rosenberger, 2012)
Furthermore, the result **P = NP** would imply many other startling results that are currently believed to be false, such as **NP = co-NP** and **P = PH**.

7.2 Consequences of P = NP

A proof that P = NP could have stunning practical consequences if the proof leads to efficient methods for solving some of the important problems in NP. It is also possible that a proof would not lead directly to efficient methods, perhaps if the proof is non-constructive, or the size of the bounding polynomial is too big to be efficient in practice. The consequences, both positive and negative, arise since various NP-complete problems are fundamental in many fields.

Cryptography, for example, relies on certain problems being difficult.

A constructive and efficient solution] to an NP-complete problem such as **3-SAT** would break most existing cryptosystems. These would need to be modified or replaced by information-theoretically secure solutions not inherently based on P-NP equivalence. (De, Kumarasubramanian, & Venkatesan, 2007) On the other hand, there are enormous positive consequences that would follow from rendering tractable many currently mathematically intractable problems. For instance, many problems in operations research are NP-complete, such as some types of integer programming and the *travelling salesman problem.* Efficient solutions to these problems would have enormous implications for logistics. Many other important problems, such as some problems in protein structure prediction, are also NP-complete; (Berger & Leighton, 1998) if these problems were efficiently solvable it could spur considerable advances in **life sciences** and **biotechnology**.

7.3 Consequences of P $\neq$ NP

A proof that showed that P $\neq$ NP would lack the practical computational benefits of a proof that P = NP, but would nevertheless represent a very significant advance in computational complexity theory and provide guidance for future research. It would allow one to show in a formal way that many common problems cannot be solved efficiently, so that the attention of researchers can be focused on partial solutions or solutions to other problems. Due to widespread belief in P $\neq$ NP, much of this focusing of research has already taken place. (Foulds, 1983)

8 Some other classes

So why has it been so hard to prove P vs. NP on way or the other? *Well, fun fact, proving things is an NP problem.* The P vs. NP question itself **is** one of these problems. So yeah, this might be difficult, or not? we don't know. As the field of computational complexity has developed, we've discovered a lot of complexity. The P vs. NP question turns out to be just the main attraction in a huge and complicated *"zoo"* of complexity classes. Beyond NP there are even harder classes of problems like **"EXP"** – the class of problems including figuring out the best move in Chess, that takes exponential time to even check. This whole upper area of problems that are at least as hard as **NP-complete** is called is called **"NP-hard"**. There's also **"Co-NP"** – the

class of problems where instead of being easy to check right answers, it's easy to exclude wrong answers, which may or may not be the same of NP. And then there's **"P-SPACE"**, the class of problems that can be solved given unlimited time, but using only a polynomial amount of space for memory. There are also problems that can be solved probabilistically in polynomial time. That class is called **"BPP"**, and it may or may not actually be the same as P. And then there's a quantum computing analog of **BPP** called **BQP**. All over the place in here there are complicated little classes that would take a lot of explaining, and, actually some of these turn out to be infinite hierarchies of problems that are slightly more difficult from the ones beneath them. We know there's an exponential hierarchy and there's probably a polynomial hierarchy. *And out beyond all of this are problems that are, just not solvable by any computer in any amount of time or space.*

9 Conclusion

To me, the amazing thing about this whole complexity zoo is that we're talking literally about what can be computed in a given amount of space, and time. We're not just looking at the nature of computation here, we're looking at the nature of space and time themselves. This mess of computational complexity classes, I think, ultimately has implications for physics and biology and, for our basic understanding of everything. As an example of those implications, here's how **Scott Aaronson**, a complexity researcher at **MIT**, explains his intuition about P vs. NP: *"If P were equal to NP, then the world would be a profoundly different place than we usually assume it to be. There would be no special value in "creative leaps", no fundamental gap between solving a problem and recognizing the solution once it's found. Everyone who could appreciate a symphony would be Mozart; Everyone who could follow a set-by-step argument would be Gauss."* The world around us, the nature of living things and ideas, of art and genius, is molded around the deep structure of computation. In a very real way, something connected to **P vs. NP** shows up in the struggles of scientists and artists. Aside from being an important problem in computational theory, a proof either way would have profound implications for **mathematics**, **cryptography**, **algorithm research**, **artificial intelligence**, **game theory**, **multimedia processing**, **philosophy**, **economics** and many other fields. (Fortnow, 2013)

References

Berger, B., & Leighton, T. (1998). Protein folding in the hydrophobic-hydrophilic (hp) model is np-complete. *Journal of Computational Biology*, *5*(1), 27–40.

De, D., Kumarasubramanian, A., & Venkatesan, R. (2007). Inversion attacks on secure hash functions using sat solvers. In *International conference on theory and applications of satisfiability testing* (pp. 377–382).

Fortnow, L. (2009). The status of the p versus np problem. *Communications of the ACM*, *52*(9), 78–86.

Fortnow, L. (2013). *The golden ticket: P, np, and the search for the impossible*. Princeton University Press Princeton.

Foulds, L. R. (1983). The heuristic problem-solving approach. *Journal of the Operational Research Society*, *34*(10), 927–934.

Gasarch, W. I. (2002). The p=? np poll. *Sigact News*, *33*(2), 34–47.

Ladner, R. E. (1975). On the structure of polynomial time reducibility. *Journal of the ACM (JACM)*, *22*(1), 155–171.

Rosenberger, J. (2012). *P vs. np poll results.* ASSOC COMPUTING MACHINERY 2 PENN PLAZA, STE 701, NEW YORK, NY 10121-0701 USA.